

Student Meal Planner

Version Control Evidence

A Managing Document for the Grubs4Scrubs Project

Date: April 2026

Author: Yao Cheng Zhou

How I Use Git

The Grubs4Scrubs project lives in a GitHub repository. I use Git for version control on both the frontend (React + Vite) and backend (ASP.NET Core). This document explains my workflow: how I commit, how I branch, and what the repo structure looks like.

[NOTE: Add your GitHub repo URL here. Also add screenshots of your commit history and any branch graphs if you have them.]

Repository Structure

The repo contains both the frontend and backend in a single repository. The folder structure looks like this:

```
Grubs4Scrubs/
├── Frontend/                                ← React + Vite frontend
│   └── src/
│       ├── Pages/                          ← Page components + CSS
│       └── Components/                     ← Shared components (Navbar, Footer,
Searchbar)
│           ├── services/                   ← Axios API service files
│           └── main.jsx
├── Grubs4Scrubs.sln                         ← .NET solution file
├── Grubs4Scrubs.API/                       ← Controllers (Recipe, User, ShoppingItem,
Auth), Program.cs
├── Grubs4Scrubs.Business/                  ← Service interfaces + implementations
├── Grubs4Scrubs.DataAccess/               ← ADO.NET repositories, DbSeeder
├── Grubs4Scrubs.Domain/                   ← Models, repository interfaces, DTOs
├── Grubs4Scrubs.Tests/                    ← Unit tests with FakeRepository
└── Grubs4Scrubs.Docs/                     ← Portfolio documentation (markdown)
```

Branching Strategy

[NOTE: Describe your actual branching approach. If you work mostly on main, say so honestly. If you use feature branches, list some examples. Here's a template for either approach:]

For a solo project, I kept it simple. Most development happened on the main branch. When I worked on bigger changes that might break things (like the Tailwind removal or the layered architecture migration), I created a feature branch so I could test everything before merging back.

Example branches: - `feature/remove-tailwind` for the CSS migration - `feature/layered-architecture` for the backend restructuring - `feature/recipe-view-page` for building the recipe detail page

[Update this with your actual branch names, or remove the feature branch section if you worked directly on main.]

Commit Practices

I try to commit after completing a logical chunk of work, not after every line change. A commit should represent one thing: "added Recipe View page", "removed Tailwind from Dashboard", "moved models to Domain project". Not "various changes" or "stuff".

[NOTE: Add 5-10 real commit messages from your history here. Something like:]

Example commits from the project:

- "Add DashboardPage with custom CSS replacing Tailwind"
- "Create RecipeViewPage with ingredient checklist"
- "Wire recipe card navigation from RecipesPage to RecipeView"
- "Split backend into layered architecture (Domain, DataAccess, Business, API)"
- "Fix namespace errors after project restructure"
- "Remove Tailwind dependencies from MealPlannerPage"
- "Add RecipeRepository with ADO.NET implementation"
- "Register DI services in Program.cs"
- "Add ShoppingItem backend (model, repo, service, controller)"
- "Add inline editing to ShoppingListPage"
- "Add AuthController with register and login endpoints"
- "Add JWT middleware configuration to Program.cs"
- "Add recipe edit modal to RecipeViewPage"

[NOTE: Swap in your actual commit messages from `git log --oneline`. These are based on work you've done but the exact wording might differ.]

What's in .gitignore

Standard ignores for both React and .NET: - `node_modules/` (frontend dependencies, reinstalled with `npm install`) - `bin/` and `obj/` (.NET build output) - `.vs/` (Visual Studio settings) - `appsettings.Development.json` (local config that might contain connection strings)

Lessons Learned About Version Control

The layered architecture migration taught me the most about Git. I moved files between projects, renamed namespaces, and updated references all in one go. That made for a messy diff. If I did it again, I'd commit after each step: first move the models, commit. Then create the DataAccess project, commit. Then move the repositories, commit. Smaller commits make it way easier to find where something broke if the build fails.

The Tailwind removal was similar. I should've committed after converting each page instead of converting everything and committing once. When something broke, I had to search through a huge diff to find the issue.

GitHub Features Used

[NOTE: Fill in whichever of these you actually use:]

- **Issues:** [Did you use GitHub Issues to track bugs or features?]
- **Projects board:** [Did you use GitHub Projects as a kanban/sprint board?]
- **Pull requests:** [Did you use PRs even as a solo dev, or merge directly?]
- **Actions/CI:** [Did you set up any GitHub Actions?]

Even for a solo project, having the code on GitHub means I have a backup, a history of every change, and a link I can share with my coach for feedback.